Empirical Study of Refresh-Token Rotation and Reuse Detection Under Concurrent Requests

Albena Veseli, Ardit Hyseni, Erblin Syla
Group 9 - Information Security
University of Prishtina | Faculty of Electrical and Computer Engineering
Email: albena.veseli@student.uni-pr.edu, ardit.hyseni5@student.uni-pr.edu, erblin.syla@student.uni-pr.edu

I. Domain and Selected Topic

The selected research domain is **security protocols, authentication, and cryptography**, with a focus on the security of **JWT, OAuth 2.0, and OpenID Connect** systems.

More specifically, the project will study how OAuth authorization servers handle **refresh-token rotation and reuse detection** when refresh requests are made concurrently. The study will compare different token rotation strategies in a controlled environment and examine how they behave when a refresh token has been compromised and is used by both a legitimate client and an attacker.

We selected this topic because it combines a well-defined, standardized security mechanism (RFC 9700) with a concrete implementation-level question that remains empirically unexplored, making it both academically grounded and technically feasible to investigate within one semester.

II. Motivation and Problem

Refresh tokens allow applications to maintain long-lived user sessions without keeping access tokens valid for long periods. However, if a refresh token is stolen, an attacker may use it to obtain new access tokens and continue accessing resources on behalf of the user.

OAuth 2.0 Security Best Current Practice (RFC 9700) recommends refresh-token rotation and reuse detection as a way of detecting the replay of compromised refresh tokens [1]. The possibility of refresh-token compromise and replay has also been discussed in the OAuth 2.0 threat model [2]. With rotation, a new refresh token is issued after a successful refresh and the previous token is no longer accepted. Reuse of an already-invalidated token can then indicate that the token has been compromised.

While RFC 9700 establishes refresh-token rotation and reuse detection as recommended security measures, the practical behavior of these mechanisms under concurrent refresh requests is implementation-dependent. In particular, the interaction between token-state checking, state updates, and concurrent requests can affect whether reuse is detected consistently. That this is not merely hypothetical is shown by CVE-2026-1035 in Keycloak, a time-of-check-to-time-of-use flaw in which concurrent requests could exchange one refresh token for several access tokens before the usage state was updated, bypassing single-use enforcement. More generally, a legitimate client and an attacker may send requests using the same refresh token at nearly the same time, and the resulting behavior may depend on whether the implementation performs a simple read-then-write operation or uses an atomic state transition.

To the best of our knowledge, no peer-reviewed empirical study has systematically measured this behavior across real implementations. Recent formal and specification-level treatments of refresh-token rotation explicitly leave this empirical characterization as open work, noting that they make no empirical claim about how frequently such race conditions occur in deployed systems [3]. The foundational formal analysis of OAuth [4] deliberately excludes token expiration and revocation, while fuzzing frameworks such as OAuthFuzz [5] aim to discover unknown logic bugs rather than to measure a specific mechanism under concurrency.

This project therefore focuses on an empirical question: how consistently do different implementations enforce refresh-token reuse detection under controlled race conditions, and what security and performance trade-offs result from different approaches?

III. Project Idea and Research Question

The main research question is:

> *How do different refresh-token rotation strategies affect reuse detection, legitimate-session survival, and performance under concurrent refresh requests?*

A secondary question is whether a configured grace window (a short interval during which a rotated token is still accepted) enables a *silent takeover*, in which an attacker gains a valid token family without any reuse alert being raised.

To investigate these questions, we will first implement a small authentication service that supports three refresh-token strategies:

- **Static refresh tokens:** the same refresh token remains valid after a successful refresh and can be reused.
- **Naive rotation:** a refresh token is replaced after successful use, using a non-atomic read-then-write state update.
- **Atomic rotation:** the token state is updated using an atomic compare-and-set operation, so that only one con-

current request can successfully consume the same refresh token.

The first strategy will serve as the baseline, while the other two will allow us to examine how the implementation of rotation affects behavior under concurrency.

In addition to the controlled implementation, we plan to test selected open-source OAuth authorization servers, such as **Keycloak** and **Ory Hydra**, running locally in Docker. This will allow us to compare the behavior of real implementations with the controlled strategies developed for the experiment.

IV. Experimental Setup

A Python-based test harness will simulate two independent clients:

- a legitimate client that owns the refresh token; and
- an attacker who has obtained a copy of the same token.

The test harness will generate refresh requests with controlled timing. The main scenarios will include:

- attacker refreshes before the legitimate client;
- legitimate client refreshes before the attacker;
- both requests are sent concurrently to create a race condition;
- the old refresh token is reused after a successful rotation.

The same scenarios will be repeated across the implemented strategies and, where supported, across the selected open-source authorization servers.

V. Measurements and Evaluation

For each configuration, we will collect both security and performance measurements.

Security measurements will include the number of successful attacker refreshes, the number of access tokens obtained after compromise, whether a single refresh token yields two or more valid successors (*family forking*), whether reuse is detected, the time required to detect reuse, and whether the legitimate session remains usable or is revoked after detection.

Performance measurements will include refresh-request latency (p50, p95, and p99), throughput, and, where practical, CPU and memory usage.

Concurrent scenarios will be repeated multiple times to determine whether the observed behavior is consistent or whether different outcomes occur depending on request timing; race outcomes will be reported as a success rate with a confidence interval rather than as a single pass or fail.

VI. Implementation and Feasibility

The complete experimental environment will run locally using free and open-source software. The controlled authentication service and test harness will be containerized with Docker Compose. The authorization servers selected for comparison will also run locally in Docker.

The project does not require cloud infrastructure or paid services, which makes the experiment feasible and allows the group to repeat the tests under the same controlled conditions.

The implementation will be kept focused on the components necessary for the experiment: an authentication service, refresh-token state management, the three rotation strategies, a concurrency test harness, and a small number of real authorization servers for comparison.

Before the main evaluation, a small pilot experiment will be performed to verify that the test harness can reliably generate concurrent requests and correctly record the resulting token states and security outcomes. Where feasible, reproducing a known defect such as CVE-2026-1035 on an unpatched server version will serve as an additional validation that the harness can observe genuine race conditions.

We anticipate that reliably triggering true race conditions (the fully concurrent scenario) may require careful timing control via asynchronous request bursts, since the vulnerable window can be sub-millisecond and the outcome is inherently probabilistic. If this proves inconsistent, the naive-vs-atomic comparison under the sequential scenarios already provides a complete and reproducible basis for the evaluation, so project completion does not depend on successfully triggering the race condition.

VII. Expected Limitations

The results will depend on the specific versions, configurations, and database or storage mechanisms used by the tested systems. Therefore, the findings will not be treated as universal properties of all OAuth authorization servers.

The project assumes that a refresh token has already been compromised and does not investigate how the token was stolen. Attack techniques such as XSS, malware, or device compromise are outside the scope of the experiment. All testing will target only the group's own local systems with synthetic clients; no real users or third-party services are involved, and any new defect found will be handled through responsible disclosure.

Concurrency results may also be affected by the operating system, hardware, network configuration, and scheduling of requests. These environmental details will therefore be documented to make the experiments as reproducible as possible.

References

[1] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best Current Practice for OAuth 2.0 Security," RFC 9700, IETF, Jan. 2025. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9700.html

[2] T. Lodderstedt, M. McGloin, and P. Hunt, "OAuth 2.0 Threat Model and Security Considerations," RFC 6819, IETF, Jan. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6819.html

[3] M. Teodori, "NEBULA: A Language-Independent Specification for Opaque Rotating Refresh Tokens," arXiv:2608.04115, Aug. 2026. [Online]. Available: https://arxiv.org/abs/2608.04115

[4] D. Fett, R. Küsters, and G. Schmitz, "A Comprehensive Formal Security Analysis of OAuth 2.0," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016, pp. 1204–1215.

[5] G. Zhang *et al.*, "OAuthFuzz: Protocol-Aware Adaptive Fuzzing for OAuth 2.0 Implementations," preprint, SSRN 7094132, 2026.